

מבחן סופי - מבוא לבטיחות תוכנהשאלת eBPF: מספר 5.1

1. Experiment with `bpftool btf dump map` and `bpftool btf dump prog` to see the BTF information associated with maps and programs, respectively. Remember that you can specify individual maps and programs in more than one way.

פתרון:

נריץ את הקוד `hello_map.py` מפרק 2 כדי ליצור מפה ותהליך BPF חדש:

```
(hacker@kali)-[~/Downloads/learning-ebpf/chapter2]
$ sudo python3 hello-map.py
ID 1000: 12
```

נריץ `sudo bpftool map list` כדי למצוא את המזהים של המפה שיצרנו בקוד:

```
(hacker@kali)-[~/Downloads/learning-ebpf/chapter2]
$ sudo bpftool map list
11: hash_of_maps name cgroup_hash flags 0x0
    key 8B value 4B max_entries 2048 memlock 169792B
17: hash name counter_table flags 0x0
    key 8B value 8B max_entries 10240 memlock 931648B
    btf_id 130
```

נריץ `sudo bpftool btf dump map id 17` כדי לראות את המאטא דאטה של המפה.
ניתן לראות בפלט ה-BTF שגם המפתח וגם הערך של המפה מוגדרים כ-64bit. זה תואם לכך שהמפה `counter_table` ממפה UID ל-Counter.

```
(hacker@kali)-[~/Downloads/learning-ebpf/chapter2]
$ sudo bpftool btf dump map id 17
[1] TYPEDEF 'u64' type_id=2
[1] TYPEDEF 'u64' type_id=2
```

נציג את המאטא דאטה של המפה גם עם Name כדי להראות שיטה נוספת להצגת הטבלה:

```
(hacker@kali)-[~]
$ sudo bpftool btf dump map name counter_table
[1] TYPEDEF 'u64' type_id=2
[1] TYPEDEF 'u64' type_id=2
```

נריץ `sudo bpftool prog list` כדי למצוא את המזהים של התהליך שרץ, בתמונה ניתן לראות את התהליך הרלוונטי ובפקודה הזו רואים את `map_ids` מהתמונה הקודמת.

```
57: kprobe name hello tag de24cf185ee252cd gpl
    loaded_at 2026-02-15T15:23:41+0200 uid 0
    xlated 208B jited 136B memlock 4096B map_ids 17
    btf_id 130
```

נריץ `sudo bpftool btf dump prog id 57` כדי לראות את המאטא דאטה של `prog`:

```
(hacker@kali)-[~/Downloads/learning-ebpf/chapter2]
$ sudo bpftool btf dump prog id 57
[1] TYPEDEF 'u64' type_id=2
[2] TYPEDEF '__u64' type_id=3
[3] INT 'unsigned long long' size=8 bits_offset=0 nr_bits=64 encoding=(none)
[4] STRUCT '___btf_map_counter_table' size=16 vlen=2
    'key' type_id=1 bits_offset=0
    'value' type_id=1 bits_offset=64
[5] INT '(anon)' size=4 bits_offset=0 nr_bits=32 encoding=(none)
[6] PTR '(anon)' type_id=0
[7] FUNC_PROTO '(anon)' ret_type_id=8 vlen=1
    'ctx' type_id=6
[8] INT 'int' size=4 bits_offset=0 nr_bits=32 encoding=SIGNED
[9] FUNC 'hello' type_id=7 linkage=static
```

נציג את התהליך גם עם `Tag` כדי להראות שיטה נוספת להצגת המאטא דאטה התהליך:

```
(hacker@kali)-[~]
$ sudo bpftool btf dump prog tag 08b4624478dd79b6
[1] TYPEDEF 'u64' type_id=2
[2] TYPEDEF '__u64' type_id=3
[3] INT 'unsigned long long' size=8 bits_offset=0 nr_bits=64 encoding=(none)
[4] STRUCT '___btf_map_counter_table' size=16 vlen=2
    'key' type_id=1 bits_offset=0
    'value' type_id=1 bits_offset=64
[5] INT '(anon)' size=4 bits_offset=0 nr_bits=32 encoding=(none)
[6] PTR '(anon)' type_id=0
[7] FUNC_PROTO '(anon)' ret_type_id=8 vlen=1
    'ctx' type_id=6
[8] INT 'int' size=4 bits_offset=0 nr_bits=32 encoding=SIGNED
[9] FUNC 'hello' type_id=7 linkage=static
```

שאלת ריברסינג (תיקיה 2):

הסיסמה של StephanKing היא Carrie.

השתמשנו בכלי Ghidra שאפשר לנו לראות את הקוד ב-C כפי שנכתב במקור. יצרנו פרויקט חדש ונכנסנו לפונקציית ה-main של StephanKing דרך ה-Symbol Tree שבתוכנה:

```
Decompile: main - (StephanKing)
1
2 undefined8 main(int param_1, long param_2)
3
4 {
5     long lVar1;
6     size_t sVar2;
7     int local_24;
8
9     if (param_1 != 2) {
10         /* WARNING: Subroutine does not return */
11         exit(1);
12     }
13     lVar1 = ptrace(PTRACE_TRACEME, 0, 0, 0);
14     if (lVar1 != 0) {
15         puts("No Debuggers");
16         /* WARNING: Subroutine does not return */
17         exit(1);
18     }
19     local_24 = 0;
20     while( true ) {
21         sVar2 = strlen(*(char **)(param_2 + 8));
22         if (sVar2 <= (ulong)(long)local_24) {
23             puts("Well done");
24             return 0;
25         }
26         if ("Carrie"[local_24] != *(char *)((long)local_24 + *(long *)(param_2 + 8)))
27             local_24 = local_24 + 1;
28     }
29     /* WARNING: Subroutine does not return */
30     exit(1);
31 }
```

מניתוח הקוד ניתן לראות שמנגנון ההגנה בשורה 13 הוא Anti-Debugging שנועד לבדוק האם התוכנית רצה תחת דיבאגר. במידה וכן הקריאה נכשלת והתוכנית מדפיסה "No Debuggers" ויוצאת מדי. התוכנית בודקת שסופק ארגורמנט אחד בדיוק אחרת היא יוצאת. לבסוף אימות הסיסמה הוא השוואת תווים פשוטה למחרוזת הסטטית "Carrie", אם כל התווים תואמים והגענו לסוף המחרוזת התוכנית מדפיסה "Well done".

הרצה עם דיבאגר:

```
(gdb) run Carrie
Starting program: /home/hacker/Downloads/th/2/StephanKing Carrie
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
No Debuggers
```

דוגמה נוספת עם ltrace שלמדנו בביתה:

```
(hacker@kali)-[~/Downloads/th/2]
$ ltrace ./StephanKing 12345
ptrace(0, 0, 0, 0) = -1
puts("No Debuggers")
) = 13
exit(1 <no return ... >
+++ exited (status 1) +++
```

אופק בר-שלום 324161421 / מאיר שוקר 318901527

הרצה עם הסיסמה הנכונה שמצאנו:

```
└─$ ./StephanKing Carrie  
Well done
```

הסיסמה של Shakespeare היא Juliet.

נשתמש שוב ב-Ghidra כדי לראות את ה-main:

```

2 undefined8 main(int param_1, long param_2)
3
4 {
5     int iVar1;
6     size_t sVar2;
7     long in_FS_OFFSET;
8     int local_4c;
9     char local_48[40];
10    long local_20;
11
12    local_20 = *(long *)(in_FS_OFFSET + 0x28);
13    builtin_strncpy(local_48, "Jvnliy", 7);
14    local_48[7] = '\0';
15    local_48[8] = '\0';
16    local_48[9] = '\0';
17    local_48[10] = '\0';
18    local_48[0xb] = '\0';
19    local_48[0xc] = '\0';
20    local_48[0xd] = '\0';
21    local_48[0xe] = '\0';
22    local_48[0xf] = '\0';
23    local_48[0x10] = '\0';
24    local_48[0x11] = '\0';
25    local_48[0x12] = '\0';
26    local_48[0x13] = '\0';
27    local_48[0x14] = '\0';
28    local_48[0x15] = '\0';
29    local_48[0x16] = '\0';
30    local_48[0x17] = '\0';
31    local_48[0x18] = '\0';
32    local_48[0x19] = '\0';
33    local_48[0x1a] = '\0';
34    local_48[0x1b] = '\0';
35    local_48[0x1c] = '\0';
36    local_48[0x1d] = '\0';
37    local_48[0x1e] = '\0';
38    local_48[0x1f] = '\0';
39    local_48[0x20] = '\0';
40    local_48[0x21] = '\0';
41    local_48[0x22] = '\0';
42    local_48[0x23] = '\0';
43    local_48[0x24] = '\0';
44    local_48[0x25] = '\0';
45    local_48[0x26] = '\0';
46    local_48[0x27] = '\0';
47    if (param_1 != 2) {
48        /* WARNING: Subroutine does not return */
49        exit(1);
50    }
51    local_4c = 0;
52    while( true ) {
53        sVar2 = strlen((char **)(param_2 + 8));
54        if (sVar2 <= (ulong)(long)local_4c) break;
55        local_48[local_4c] = local_48[local_4c] - (char)local_4c;
56        local_4c = local_4c + 1;
57    }
58    iVar1 = strcmp((char **)(param_2 + 8), local_48);
59    if (iVar1 == 0) {
60        puts("Well done");
61    }
62    if (local_20 != *(long *)(in_FS_OFFSET + 0x28)) {
63        /* WARNING: Subroutine does not return */
64        __stack_chk_fail();
65    }
66    return 0;

```

בתוכנית זו הסיסמה לא שמורה כטקסט גלוי בזיכרון, היא עוברת תהליך פענוח בזמן ריצה.

בתחילת ה-main התוכנית טוענת ל-local_48 את המחרוזת "Jvnliy".

לאחר מכן בכל סיבוב של לולאת ה-while התוכנית מבצעת את החישוב:

$$local_48[i] = local_48[i] - i$$

כאשר i זה המיקום של התו במחרוזת. בסוף הלולאה המחרוזת הופכת בזיכרון ל- "Juliet".

החישוב שלנו:

אינדקס 0: האות 'נ' פחות 0 שווה 'נ'

אינדקס 1: האות 'ו' פחות 1 שווה 'ט'

אינדקס 2: האות 'ח' פחות 2 שווה 'ו'

אינדקס 3: האות 'ל' פחות 3 שווה 'י'

אינדקס 4: האות 'י' פחות 4 שווה 'e'

אינדקס 5: האות 'י' פחות 5 שווה 't'

לאחר מכן התוכנית משווה את ארגומנט המשתמש למחרוזת שקיבלנו ואם יש התאמה מלאה, מדפיסה

:"Well done"

```

$ ./Shakespeare Juliet
Well done

```

בנוסף ללוגיקת התוכנית, הקומפיילר שילב מנגנון הגנה מובנה נגד מתקפות Buffer Overflow.

בתחילת הפונקציה (שורה 12), התוכנית קוראת ערך אקראי וסודי אל תוך המשתנה המקומי local_20. משתנה זה ממוקם על המחסנית באופן אסטרטגי, בדיוק בין הbuffer שאליו נכתב הקלט לבין כתובת החזרה של הפונקציה. הרעיון מאחורי המנגנון הוא שמאחר שהbuffer מוגבל ל-40 תווים, תוקף שינסה להזין קלט ארוך יותר כדי לדרוס את כתובת החזרה ולהריץ קוד זדוני, ייאלץ בהכרח לדרוס בדרך גם את הערך השמור ב-local_20.

בסיום הפונקציה, מתבצעת שורת הבדיקה (62). התוכנית מוודאת שהערך הנוכחי ב-local_20 נותר זהה לערך המקורי. אם הערכים זהים, המחסנית תקינה והתוכנית מסתיימת בהצלחה.

אם מתגלה חוסר התאמה, המחסנית נדרסה. לכן התוכנית קופצת מיד לפונקציית החירום שמקריסה את התוכנית באופן יזום וחוסמת את ניסיון ההשתלטות.

הסיסמה של AgathaChristie היא Marple.

נשתמש ב-Ghidra כדי לראות את ה-main:

```

1  undefined8 main(int param_1,long param_2)
2
3
4 {
5     long lVar1;
6     size_t sVar2;
7     long in_FS_OFFSET;
8     int local_4c;
9     byte local_48 [40];
10    long local_20;
11
12    local_20 = *(long *) (in_FS_OFFSET + 0x28);
13    local_48[0] = 0xa3;
14    local_48[1] = 0x8f;
15    local_48[2] = 0x9c;
16    local_48[3] = 0x9e;
17    local_48[4] = 0x82;
18    local_48[5] = 0x8b;
19    local_48[6] = 0;
20    local_48[7] = 0;
21    local_48[8] = 0;
22    local_48[9] = 0;
23    local_48[10] = 0;
24    local_48[0xb] = 0;
25    local_48[0xc] = 0;
26    local_48[0xd] = 0;
27    local_48[0xe] = 0;
28    local_48[0xf] = 0;
29    local_48[0x10] = 0;
30    local_48[0x11] = 0;
31    local_48[0x12] = 0;
32    local_48[0x13] = 0;
33    local_48[0x14] = 0;
34    local_48[0x15] = 0;
35    local_48[0x16] = 0;
36
37    local_48[0x26] = 0;
38    local_48[0x27] = 0;
39    if (param_1 != 2) {
40        /* WARNING: Subroutine does not return */
41        exit(1);
42    }
43    lVar1 = ptrace(PTRACE_TRACEME,0,0,0);
44    if (lVar1 != 0) {
45        puts("No Debuggers");
46        /* WARNING: Subroutine does not return */
47        exit(1);
48    }
49    local_4c = 0;
50    while( true ) {
51        sVar2 = strlen((char **)(param_2 + 8));
52        if (sVar2 <= (ulong)(long)local_4c) break;
53        if ((*byte *) ((long)local_4c + *(long *) (param_2 + 8)) ^ 0xee) != local_48[local_4c]) {
54            /* WARNING: Subroutine does not return */
55            exit(1);
56        }
57        local_4c = local_4c + 1;
58    }
59    puts("Well done");
60    if (local_20 != *(long *) (in_FS_OFFSET + 0x28)) {
61        /* WARNING: Subroutine does not return */
62        __stack_chk_fail();
63    }
64    return 0;
65 }

```

בתוכנית זו הסיסמה לא מופיעה לנו כטקסט גלוי כמו בראשון או משובש

כמו בשני, היא מוחבאת באמצעות XOR.

כמו בתוכניות הקודמות גם כאן יש מנגנוני הגנה כמו אנטי-דיבאגינג ובדיקת דריסת זיכרון.

בתחילת הפונקציה התוכנית בונה מערך בתים בשם local_48 המכיל ערכים הקסדצימליים.

בלולאת ה-while התוכנית לוקחת כל תו שהזנו מבצעת עליו פעולת XOR עם הערך הקבוע 0xee ומשווה

את התוצאה לערך המתאים ב-local_48.

כדי למצוא את הסיסמה המקורית, עלינו לבצע פעולת XOR הופכית על הערכים במערך עם המפתח 0xee.

כלומר: $argv[1][i] = local_48[i] \oplus 0xee$

החישוב שלנו:

תו 1: $0xa3 \oplus 0xee = 0x4D$ האות Mתו 2: $0x8f \oplus 0xee = 0x61$ האות aתו 3: $0x9c \oplus 0xee = 0x72$ האות rתו 4: $0x9e \oplus 0xee = 0x70$ האות pתו 5: $0x82 \oplus 0xee = 0x6C$ האות lתו 6: $0x8b \oplus 0xee = 0x65$ האות e

שילוב האותיות יוצר את השם Marple:

```

(hacker@kali)-[~/Downloads/th/2]
$ ./AgathaChristie Marple
Well done

```

שאלת הזרקת ספרייה (סעיף 4):קוד התוכנית המותקפת (target.c):

תוכנית זו מדמה את קורבן התקיפה. היא פותחת את הקובץ הלגיטימי /dev/random, קוראת ממנו 4 בתים ומדפיסה את התוצאה למסך.

```

1#include <stdio.h>
2#include <stdlib.h>
3
4int main() {
5    FILE *f = fopen("/dev/random", "r");
6    if (!f) {
7        perror("Failed to open /dev/random");
8        return 1;
9    }
10
11    unsigned int secret_password;
12    printf("Reading secret password from /dev/random...\n");
13
14    fread(&secret_password, sizeof(secret_password), 1, f);
15
16    printf("The secret password is: %u\n", secret_password);
17
18    fclose(f);
19    return 0;
20}

```

קוד הספרייה הזדונית (hook_elf.c):

קוד זה מבוסס על שלד המעבדה מהמצגת במודל. מכיוון שאנחנו עורכים את קובץ ה-ELF ומוחקים את התלות שלו בספריית libc המקורית, הספרייה שלנו משתמשת בפונקציית אתחול כדי לטעון ידנית את libc לזיכרון בעזרת dlopen.

לאחר מכן, אנו דורסים את הפונקציה fread כך שתפנה לקובץ החלופי fake_random.txt.

```

1#include <stdio.h>
2#include <dlfcn.h>
3#include <stdlib.h>
4
5#define OBJ_PATH "/lib/x86_64-linux-gnu/libc.so.6"
6
7void* handle;
8size_t (*original_fread)(void*, size_t, size_t, FILE*);
9
10static void myinit() __attribute__((constructor));
11static void mydest() __attribute__((destructor));
12
13void myinit() {
14    handle = dlopen(OBJ_PATH, RTLD_LAZY);
15    if (handle) {
16        original_fread = dlsym(handle, "fread");
17    }
18}
19
20void mydest() {
21    if (handle) dlclose(handle);
22}
23
24size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream) {
25    FILE *fake_file = fopen("fake_random.txt", "r");
26    if (fake_file && original_fread) {
27        size_t result = original_fread(ptr, size, nmemb, fake_file);
28        fclose(fake_file);
29        return result;
30    }
31
32    return 0;
33}

```


תהליך העבודה ופקודות להרצה:

כדי לבצע את התקיפה בפועל, נבצע את השלבים הבאים:

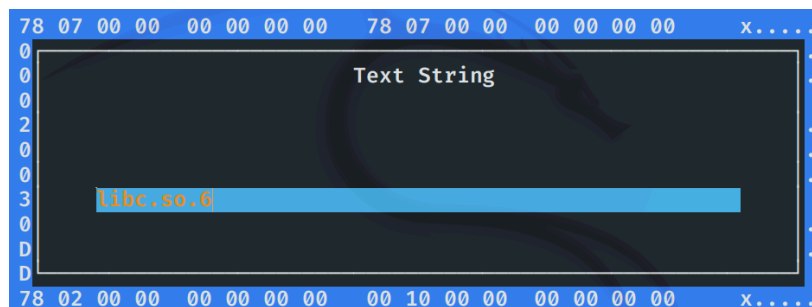
1. יצרנו את הקובץ טקסט המכיל את הסיסמה הסודית: `echo "1234" > fake_random.txt`
2. קימפלנו את התוכנית המותקפת ואת הספרייה הזדונית תחת השם `xibc.so.6`:

```
(hacker@kali)-[~/Documents]
$ gcc target.c -o target
/usr/bin/ld: error in /usr/lib/gcc/x86_64-linux-gnu/14/../../../../x86_64-linux-gnu/S
crt1.o(.sframe); no .sframe will be created

(hacker@kali)-[~/Documents]
$ gcc -shared -fPIC hook_elf.c -o xibc.so.6 -ldl
```

הערה לגבי הפלט: ההודעה שמופיעה היא אזהרה לגרסה הנוכחית של הקאלי לינוקס שהשתמשנו בה, היא לא שגיאת קוד והיא לא מונעת את יצירת הקובץ הבינארי התקין. נסביר את השימוש בדגלים ביצירת הספרייה הדינמית בקצרה: `shared`: אומר לקומפיילר ליצור ספרייה משותפת ולא קובץ הרצה רגיל. `fPIC`: חובה בבניית ספרייה דינמית כדי שהקוד יוכל להיטען לכל כתובת בזיכרון. `ldl`: אומר ללינקר לקשר את הספרייה `libdl`.

3. עריכת החלק הדינמי ב-ELF: פתיחת הקובץ הבינארי `target` בעזרת `hexeditor`, חיפוש המחרוזת `:libc.so.6`



החלפת האות הראשונה מ-`l` ל-`x`, כך שהתוכנית תדרוש את הספרייה `xibc.so.6` במקום את הספרייה הלגיטימית:

```
ASCII Offset: 0x00000579 / 0x00003F97 (%09) M
78 69 62 63 2E 73 6F 2E .printf.xibc.so.
32 2E 32 2E 35 00 47 4C 6.GLIBC_2.2.5.GL
00 5F 49 54 4D 5F 64 65 IBC_2.34._ITM_de
54 4D 43 6C 6F 6E 65 54 registerTMCloneT
6D 6E 6E 5E 73 74 61 72 able. gmon star
```

4. הפעלנו את התוכנית תוך כדי הוספת התיקיה הנוכחית לנתיב של חיפוש הספריות של מערכת ההפעלה: `LD_LIBRARY_PATH=./target`

```
(hacker@kali)-[~/Documents]
$ LD_LIBRARY_PATH=./target
./target: ./xibc.so.6: no version information available (required by ./target)
./target: ./xibc.so.6: no version information available (required by ./target)
Reading secret password from /dev/random...
The secret password is: 875770417
```

הערה לגבי הפלט: אזהרה זו נובעת מכך שקובץ ה-`target` מצפה למצוא חתימות הקיימות ב-`libc` המקורי. הספרייה המזויפת שיצרנו לא מכילה את החתימות האלו. המערכת מתריעה על זה אבל ממשיכה בהרצה, מה שמאפשר להזרקה לעבוד.

המספר שהודפס "875770417" הוא יצוג מספרי של "1234" שהזנו לקובץ ולכן הוא מוכיח שהתוכנית קראה את התוכן של fake_random.txt, מה שמאשר שההזרקה הצליחה.

שאלת פורנזיקה (מספר 6):

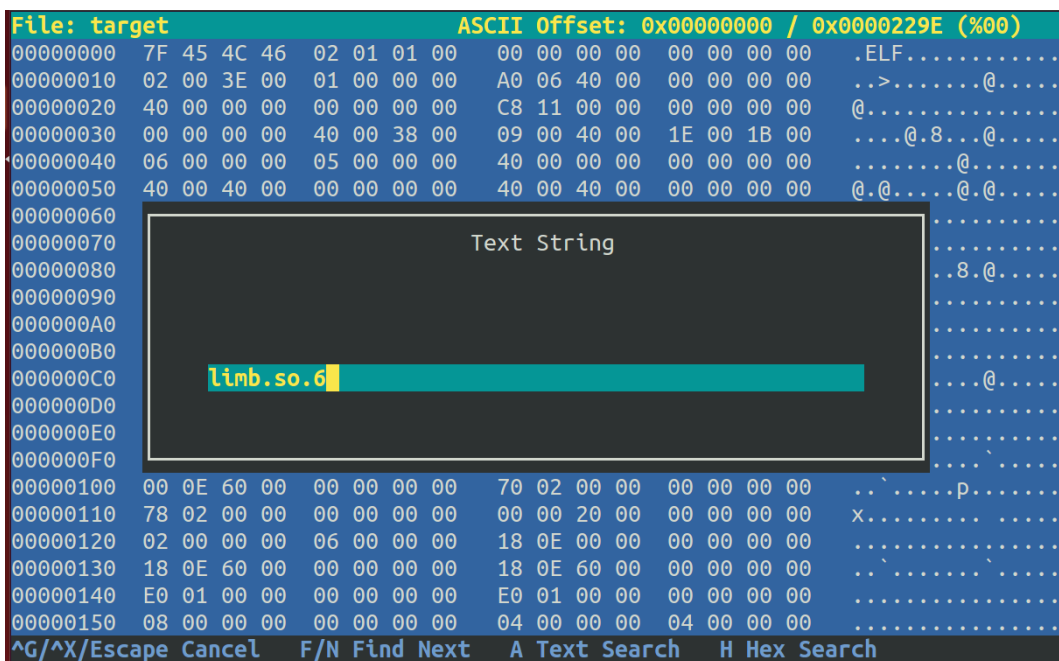
נראה באיזה ספריות משתמש התהליך של מעבדת ההזרקה ממבחן זה.
הספרייה הזדונית דורסת את הפונקציה fread ומפנה אותה לקובץ מזויף.
על מנת לאפשר צילום יציב של הזיכרון, הוספנו לקוד target פקודת getchar שתעצור את התוכנית

ובכך נוכל לצלם את הזיכרון בזמן שהתוכנית רצה:

```
printf("Press Enter to start the program and read the secret...\n");
getchar();
```

```
student@ubuntu:~/Documents$ LD_LIBRARY_PATH=. ./target
Press Enter to start the program and read the secret...
```

עריכת ה-ELF: בעזרת hexeditor שנינו את target והחלפנו את המחרוזת limb.so.6 במחרוזת xibm.so.6 ובכך אנחנו טוענים את הספרייה שלנו במקום את הספרייה הלגיטימית:



.xibm.so.6._ITM_

בשלב הזה, כשהתהליך רץ ומחכה ב-RAM, השתמשנו בכלי LiME כדי ליצור תמונת מצב מלאה של זיכרון המערכת, בנינו את המודול בעזרת make ולאחר מכן ביצענו את הצילום:

```
student@ubuntu:~/volatility/LiME/src$ make
make -C /lib/modules/3.13.0-32-generic/build M="/home/student/volatility/LiME/src" modules
make[1]: Entering directory `/usr/src/linux-headers-3.13.0-32-generic'
Building modules, stage 2.
MODPOST 1 modules
LD [M] /home/student/volatility/LiME/src/lime.ko
make[1]: Leaving directory `/usr/src/linux-headers-3.13.0-32-generic'
strip --strip-unneeded lime.ko
mv lime.ko lime-3.13.0-32-generic.ko
```

```
student@ubuntu:~/volatility$ sudo insmod LiME/src/lime-3.13.0-32-generic.ko "path=./mem_dump.bin for mat=lime"
```

לאחר מכן הסרנו את המודל כדי לחסוך במקום במערכת: `sudo rmmod lime`.

מצאנו את הPID של התהליך target בעזרת הפקודה הבאה:

```
student@ubuntu:~/volatility$ pidof target
4298
```

נעת השתמשנו ב-Volatility 2 כדי לחלץ את רשימת הספריות של התהליך מתוך קובץ ה-Dump:

```
student@ubuntu:~/volatility$ python vol.py -f mem_dump.bin --profile=LinuxUbuntu-3_13_0-32-genericx64 linux_ldrmodules -p 4298
Volatility Foundation Volatility Framework 2.6.1
Pid      Name      Start      File Path      Kern
el Libc
-----
4298 target  0x0000000000400000 /home/student/Documents/target      True
False
4298 target  0x00007fd85c7b1000 /lib/x86_64-linux-gnu/libdl-2.19.so   True
True
4298 target  0x00007fd85cd7b000 /home/student/Documents/xibm.so.6     True
True
4298 target  0x00007fd85c9b5000 /lib/x86_64-linux-gnu/libc-2.19.so     True
True
4298 target  0x00007fd85cf7d000 /lib/x86_64-linux-gnu/ld-2.19.so      True
True
```

הפלט מראה בבירור את טעינת הספרייה. הופעת הספרייה בתוך הכתובת של התהליך מוכיחה שדריסת ה-DT_NEEDED ב-ELF אכן גרמה לטעינת הקוד הזדוני לזיכרון.

שאלת בונוס XZ (סעיף 1)

מכיוון שהזיהוי של פרצת ה-XZ היה מוקדם יחסית, ההשפעה הישירה הוגבלה לגרסאות 5.6.0 ו-5.6.1 בלבד. ההפצות שנפגעו היו בעיקר גרסאות פיתוח ו-Rolling Release כגון Kali Linux, Fedora Rawhide, OpenSUSE Tumbleweed ו-Debian Testing. משתמשים וארגונים שהריצו מערכות אלו נאלצו לפרמט ולהתקין מחדש את תשתיותיהם כדי לוודא שאין דליפת מידע או התבססות של נוזקה ברמה גבוהה. הפרצה תוכננה לאפשר לתוקפים יכולת הרצת קוד מרחוק תוך עקיפה מלאה של מנגנוני האימות ב-Ssh, מה שהיה הופך מיליוני שרתים ברחבי העולם לחשופים לחלוטין.

מעבר לפגיעה הטכנית, נגרם נזק פסיכולוגי משמעותי לאמון במודל התחזוקה של הקוד הפתוח. המודל המסורתי של "קהילה שבודקת את הקוד" הועמד במבחן קשה, כשהתברר שתוקף סבלני ומתוחכם יכול להחדיר קוד זדוני מורכב ומוסווה היטב מבלי לעורר חשד במשך זמן רב. התוקף השתמש בשיטות הנדסה חברתית מתוחכמות לאורך שנתיים כדי לרכוש מעמד של מתחזק לגיטימי. אירוע זה הוכיח כמה קל לנצל את העומס המוטל על מפתחים מתנדבים כדי להחדיר נזקות לשרשרת האספקה העולמית, והוא הוביל לשינוי תפיסתי עמוק באופן שבו מדינות ותאגידים בוחנים כיום את אבטחת רכיבי הקוד הפתוח עליהם הם נשענים.

בליבליוגרפיה:

- [wikipedia akamai catonetworks](https://www.wikiwand.com/en/Kernel_vulnerability)
- <https://ebpf.io/what-is-ebpf/>
- <https://www.youtube.com/watch?v=hKNl0FvT7qU>
- <https://medium.com/@0x0Aleem/practical-memory-forensics-with-volatility-2-3-windows-and-linux-cheat-sheet-ef5eee325863>
- <https://gemini.google.com/app>: נעזרנו במודל השפה Gemini לצורך הבהרת מושגים תיאורטיים במבנה קבצי ELF, איתור דגלי קומפילציה מתאימים ופענוח הודעות שגיאה של מערכת ההפעלה במהלך ניסויי הפורנזיקה.